



Algorithm C++

created by

Abdallah Alqiran

Rana Badawy.

Ibrahim Ali

Mayar Abdelrahim

Agenda

- Mod Law
- Palindrome law in numbers
- Math
- All about summation
- Sum of numbers L,R that divide by X
- some geometry lows
- Fibonacci fast law
- Max Subarray sum
- sorting
- Binary Exponentiation (Fast Power)
- Divisors
- Prime factorization
- check is prime
- Bitmask
- All subset
- Monotonic stack
- fast pow
- Prefix Sum

- Sliding Window
- binary search
- lower_bound and upper_bound
- Monotonic Stack
- GCD, LCM
- Big Mod
- graph
- BFS (Breadth-First Search)
- DFS (Depth First Search)
- Sieve of Eratosthenes
- Different types of subs
- Ordered set
- Ordered Map

bits library, fast code and freopen

```
#include <bits/stdc++.h>

ios_base::sync_with_stdio(0);
cin.tie(0),cout.tie(0);

freopen("GG.in", "r", stdin);
```

Mod law

```
// note: A and B should be int or long long
A % B = A - (B * (A / B));
// if you have a negative number and you want to get the mod
A = (A % B + B) % B;
```

Palindrome law in numbers

```
// calculate the number of numbers that is palindrome
// according to digits num
ll numPal = 9 * pow(10, (numberOfDigits - 1) / 2);
```

Math

```
//to calculate the num of digits for number n
ll digits = log(n)/log(x)+1;
// note : x is the base of number system
```

```
//logarithm ⇒ notes
log(a*b)=log(a)+log(b)
logx(n)= log(n)/log(x)
log(nx)=x*log(n)
```

```
// -----
floor() function → number/=2;
ceil() function → ++number/=2;
```

All about summation

```
// summation of numbers
ll sum = (N) * (N + 1) / 2;

// summation of square numbers
ll sumSquare = (N)*(N + 1)*(2 * N + 1)/6;

// summation of numbers that divisible by k from 1 to n
ll sumDiv = k * (n/k * (n/k+1))/2;

// get the number from the summation
ll number = (-1 + sqrt(1+8 * sum)) / 2;

// summation (pow(X,2) - X)
ll sum = (N) * (N + 1) * (N - 1) / 3;
```

```
// sum arithmetic sequence
ll sum = (N * (2 * A + (N - 1) * D)) / 2;
// N the number of terms
// A the first term
// D the common difference
```

Sum of numbers L,R that divide by X

```
ll L,R;
// L is the min
// R is the max

ll L = (A - 1) / X;
ll R = B / X;
ll sum = ((R * (R + 1)) / 2 - (L * (L + 1)) / 2) * X;
```

some geometry lows

```
// distance between two points
double d = sqrt(pow(y2 - y1, 2) + pow(x2 - x1, 2));

// triangle
ll perimeter=a+b+c;
double area=.5*base*height;
// Heron's formula
double s=perimeter/2;
double area = sqrt(s*(s-a)*(s-b)*(s-c));
//median for triangle
double ma = 0.5 * sqrt(2*pow(b,2) + 2*pow(c,2) - pow(a,2));
// note ⇒ a/sin(A) = b/sin(B) = c/sin(C)
// c*c= a*a + b*b - 2*a*b*cos(C);

//circle
double pi=3.141592654;
double area = pi*r*r;
double perimeter = 2*pi*r;
```

```

// ball
double volume=4/3*pi*r*r*r
//cylinder
double area = 2*pi*r*(h+r)
double vol = pi*r*r*h;

// diameter of the square
double di=sqrt(2)*x;

//trapezoid (شبه المنحرف)
ll perimeter = a+b+c+d;
double area = .5 * h * (b+d);

// regular polygons
double interior angle = ((n-2) * 180)/n;
double exterior angle = 360 / n;

//It prints the number with six digits after the decimal point
cout<<fixed<<setprecision(6);

// To check if the three sides make a triangle
The sum of the two smaller sides must be greater than the third side

```

Fibonacci fast law

```

F(N)=((pow(1 + sqrt(5),N))-(pow(1-sqrt(5),N)))/(pow(2,N)*sqrt(5));

```

making array = 0

```

// for array
memset(A,0,sizeof(A));
// for vector
fill(V.begin(),V.end(),0);

```

Number of elements in array

```
int N = sizeof(A) / sizeof(A[0]);
```

Max Subarray sum

```
int arr[N];
int best = 0, sum = 0;
for (int i=0; i<N; ++i)
{
    sum = max(arr[i], sum+arr[i]);
    best = max(sum, best);
}
cout << best;
```

sorting

```
// bubble sort O(N^2)
for (int i=0; i<N; ++i)
{
    for (int j=0; j<N-1; ++j)
    {
        if (arr[j] > arr[j+1])
            swap(arr[j], arr[j+1]);
    }
}
// ----- //
// selection sort O(N^2)
int mn;
for(int i=0; i<n-1; i++)
{
    mn=i;
    for(int j=i+1; j<n; j++)
    {
        if(arr[j]<arr[mn])
            mn=j;
    }
    swap(arr[mn], arr[i]);
}
```

```

}
// ----- //
// Quick sort ( best  $\Rightarrow O(n \cdot \log(n))$  | worst  $\Rightarrow O(n^2)$ 

// ----- //
bool precede(int x, int y)
{
    return x > y; // this will sort from greater to smaller
}

int main()
{
    sort(a, a+n, precede);
}

// ----- //
// Insertion Sort
void insertionSort(int arr[], int N)
{
    int key, j;

    for (int i=0; i<N; ++i)
    {
        key = arr[i];
        j = i - 1;

        while(j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
// In this type you start from index 1

```

```
// and see where you should put the number you stand for
// to make it sort
```

sort array of objects

```
// using operator overload
//////////struct with operator overload //////////
struct student{
    string name;
    int a,b,c,d;
    int sum(){
        return a+b+c+d;
    }
    // we overload < to make sort function call this
    //because sort use this operator (<) in sorting
    bool operator<( student& other){
        if (sum() != other.sum()) {
            return sum() > other.sum();
        }
        return name < other.name;
    }
};
////////// main code //////////
int main()
{
    int n;
    cin>>n;
    vector<student>v(n);
    for(int i=0;i<n;i++){
        cin>>v[i].name>>v[i].a>>v[i].b>>v[i].c>>v[i].d;
    }
    sort(v.begin(),v.end());
    // we already overload operator (<)
    //to make it sort according to score and name
    for(int i=0;i<n;i++){
        cout<<v[i].name<<" "<<v[i].sum()
```



```

        <<" "<<v[i].a<<" "<<v[i].b<<" "
        <<v[i].c<<" "<<v[i].d<<"\n';
    }
}

////////////////////////////////////

// using compare function
////////// struct //////////
struct student{
    string name;
    int score;
};
////////// compare function //////////
bool comp(student & a , student & b){
    if(a.score != b.score){
        return a.score>b.score;
    }
    else{
        return a.name<b.name;
    }
}
////////// code in main //////////
int main()
{
    int n;
    cin>>n;
    vector<student>v(n);
    for(int i=0;i<n;i++){
        cin>>v[i].name>>v[i].score;
    }
    sort(v.begin(),v.end(),comp); // to use sort with compare function
    for(int i=0;i<n;i++){
        cout<<v[i].name<<" "<<v[i].score<<"\n';
    }
}

////////////////////////////////////

```

Binary Exponentiation (Fast Power)

```
ll power(int x, int p)
{
    ll ans = 1;
    while (p) // mean if p > 0
    {
        if (p & 1) // the number is odd same as p % 2 == 1
            ans *= x;
        x *= x;
        p /= 2; // same as p >> 1;
    }
    return ans;
}

// Modular & binary exponentiation >> famous problem
// time o(log2(n))
ll modExp(int x, int n, int m){
    int res=1;
    while(n>0){
        if(pow%2 == 1) res = (res*x)%m;
        x = ((x%m)*(x%m))%m;
        n/=2;
    }
    return res;
}
```

this code will get the power of number with less time ($\log(N)$)

Divisors

```
int N;    cin >> N;
vector<ll> V;

for (int i = 1; i <= N / i; ++i)
{
    if (N % i == 0)
    {
```

```

        V.push_back(i);
        if (i * i != N)
            V.push_back(N / i);
    }
}

```

Prime factorization

```

int N;
cin >> N;

for (int i = 2; i <= N / i; i += 1 + (i & 1))
{
    while(N % i == 0)
    {
        cout << i << ' ';
        N /= i;
    }
}
if (N > 1)
    cout << N;

```

This will get all prime numbers that divided by N

check is prime

```

bool is_prime(int N)
{
    if (N % 2 == 0 || N <= 1)
        return 0;
    if (N == 2)
        return 1;
    for (int i = 3; i <= N / i; i += 2)
    {
        if (N % i == 0)
            return 0;
    }
}

```

```

        return 1;
    }
    //time: O(sqrt(N))

    //////////////////////////////////////

    // Sieve algorithm
    const NN = 10e6 + 7;
    bool is_prime[NN];
    void sieve()
    {
        memset(is_prime, true, sizeof is_prime);
        is_prime[0] = is_prime[1] = false;

        for (int i = 2; i * i <= NN; ++i)
        {
            if (is_prime[i]) // means the number is prime
            {
                for (int j = i*i; j < NN; j += i)
                    is_prime[j] = false;
            }
        }
    }
}

```

Bitmask

```

// built in function

// this will tell you how many 1 you have
// in binary representation of the number
__builtin_pop(n);
////////////////////////////////////

    int i,n; // i is the number of the bit you want to do operation
            // n is the main number
// to know the bit is 1 or 0
bool knowbit(int n,int i)
{

```

```

        return (n >> i) & 1;    // shift right
    }

    // make the bit = 1
    int setbit(int n,int i)
    {
        return (n|(1 << i));    // shift left
    }

    // make the bit = 0
    int resetbit(int n,int i)
    {
        return (n & ~(1 << i));
    }

    // to flip the bit
    int flip(int n, int i)
    {
        return (n ^ (1 << i));
    }

    // know if the number is power of two or not
    bool isPowerOfTwo(int n, int i)
    {
        if (n == 0)
            return 0;
        return ~(n & (n - 1))
    }

    // convert to binary
    string FromDisToBin()
    {
        string S = "";
        while (N > 0)
        {
            S += (N % 2 + '0');
            N /= 2;
        }
    }

```

```
}  
}
```

All subset

```
bool KnowBit(int N,int i)  
{  
    return (N >> i) & 1;  
}  
  
void solve()  
{  
    int N;    cin >> N;  
  
    vector<ll> V(N);  
    vector<vector<ll>> ans;  
  
    for (auto &i : V)  
        cin >> i;  
  
    for (int i=0;i<(1<<N);++i)  
    {  
        vector<ll> Vout;  
  
        for (int j=0;j<N;++j)  
        {  
            if (KnowBit(i,j) == 1)  
                Vout.push_back(V[j]);  
        }  
  
        ans.push_back(Vout);  
    }  
    sort(ans.begin(),ans.end());  
  
    for (int i=0;i<ans.size();++i)  
    {  
        for (int j=0;j<ans[i].size();++j)
```

```

        {
            cout << ans[i][j] << ' ';
        }
        cout << '\n';
    }
}

```

subset law is $(\text{pow}(2, N) - 1)$

Monotonic stack

```

vector<ll> V, V2;
stack st;

for (int i=0; i<N; ++i)
{
    while(!st.empty() && st.top() < V[i])
    {
        st.pop();
    }

    if (st.empty())
    {
        result[i] = st.top();
    }
    else
    {
        result[i] = -1;
    }
    st.push(V[i])
}

```

fast pow

```

ll fast_pow(ll x, ll p)
{
    ll ans = 1;

```

```

while (p)
{
    if (p & 1) ans *= x;
    x *= x;
    p >>= 1;
}
return ans;
}

ll fast_pow_mod(ll x, ll p, ll m)
{
    ll ans = 1;
    while (p)
    {
        if (p & 1) ans = (ans * x) % m;
        x = (x * x) % m;
        p >>= 1;
    }
    return ans;
}

```

fast pow

```

__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

void print(__int128 x) {
    if (x < 0) {

```



```

        putchar('-');
        x = -x;
    }
    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}

```

when you put this in your code it doesn't overflow, but you should make print() and don't use the fast code

Prefix Sum

```

// Enter array to get the range sum
int N;    cin >> N;

vector<long long> V(N), prefix(N);

for (int i=0;i<N;++i)
{
    cin >> V[i];
    if (i == 0)
        prefix[i] = V[i];
    else
        prefix[i] = prefix[i-1] + V[i];
}

// Print sum of range [L, R]
int L, R;    cin >> L >> R;
// if base 1 → L--, R--;
long long sum = prefix[R] - prefix[L-1];

cout << sum;

```

Sliding Window

Get max sum of subarray with size K

```

int N, K;    cin >> N >> K;
vector<int> V(N);

for (auto &i: V)
    cin >> i;

int sum = 0;
for (int i=0;i<K;++i)
    sum += V[i];

int mxSum = sum;
for (int i=K;i<N;++i)
{
    sum += V[i] - V[i-K];
    mxSum = max(mxSum, sum);
}

cout << mxSum;

```

binary search sort before

```

///// dont forget to sort /////
int binarySearch(vector<ll> V, int element)
{
    int l = 0, r = size - 1;
    int mid;

    while (l <= r)
    {
        // mid = l + (r - l) / 2;
        // the same but better
        mid = l + (r - l) / 2;
        if (V[mid] == element)
            return mid;
        else if (V[mid] > element)
            r = mid - 1;
        else

```

```

        l = mid + 1;
    }
    return -1;
}

```

Recursive approach

```

int binarySearch(vector<int> nums, int left, int right, int target) {

    int mid = left + (right - left) / 2;

    if (left > right)
        return -1;
    if (nums[mid] > target) {
        return binarySearch(nums, left, mid-1, target);
    } else if (nums[mid] < target) {
        return binarySearch(nums, mid+1, right, target);
    }

    return mid;
}

```

Time Complexity: $O(\log_2(n))$

lower_bound **sort before**

```

///// dont forget to sort /////
vector<int> V(N);
int number;    cin >> number;

int l=0, r = N-1;
int mid;
int answer = -1;

while (l <= r)
{
    mid = l + (r - l) / 2;
}

```

```

    if (V[mid] >= number)
    {
        answer = V[mid];
        r = mid - 1;
    }
    else
        l = mid + 1;
}
cout << answer;
////////////////////
// if you want to print the number
cout << *lower_bound(V.begin(),V.end(),number);
// or
auto it = lower_bound(V.begin(),V.end(),number);
cout << *it;

```

lower_bound : bigger than or equal

Upper_bound : bigger than only

if you want to get upper just put equal in this code

Monotonic Stack

```

int N;
cin >> N;
vector<ll> V1(N), V2(N,0);
stack<ll> S;

for (int i=0;i<N;++i)
    cin >> V1[i];

for (int i=0;i<N;++i)
{
    while(!S.empty() && V1[i] > V1[S.top()])
    {
        V2[S.top()] = i + 1;
        S.pop();
    }
}

```

```

        S.push(i);
    }

    for (int i=0;i<N;++i)
    {
        if (V2[i] == 0)
            V2[i] = -1;
    }

```

This algorithm finds the next greater element for each item in the array.

GCD, LCM

```

// solution
// GCD (// A, // B)
{
    // temp;
    while (B!=0)
    {
        temp = A;
        A = B;
        B = temp % A;
    }
    return A;
}

// another solution

// GCD (// A, // B)
{
    while (A != 0 && B != 0)
    {
        if (A >= B)
            A %= B;
        else
            B %= A;
    }
    return max(A,B);
}

```

```

}

// with built in function

ll A,B;
cout << __gcd(A,B);

// LCM law

A * B / __gcd(A,B);

```

GCD → Greatest Common Divisor.

LCM → Least Common Multiple.

Big Mod

```

ll BigMod(string S, ll M)
{
    ll sum = 0;
    for (int i=0;i<S.size();++i)
    {
        sum = (sum * 10 + (S[i] - '0')) % M;
        if (sum == 0)
            return sum;
    }
    return sum;
}

```

graph

Adjacency List

```

int N,M;    cin >> N >> M;
vector<vector<ll>> adj(M+1);
for (int i=0;i<M;++i)
{
    int a,b;
    cin >> a >> b;
}

```

```
adj[a].push_back(b);
// if the graph directed remove next line
adj[b].push_back(a);
}
```

Space complexity $O(2 \cdot E) \rightarrow E$ is the total number of edges

Weighted Adjacency List

graph have a value in it's edge

```
int N, M;    cin >> N >> M;
vector<vector<pair<int,int>>> adj(M+1);
for (int i=0;i<M;++i)
{
    int a, b, weight;
    cin >> a >> b >> weight;
    adj[a].push_back({b,weight});
    // if the graph directed remove next line
    adj[b].push_back({a,weight});
}
```

Adjacency Matrix

```
int N,M;    cin >> N >> M;
vector<vector<ll>> adj(M+1,vector<ll> (M+1));
for (int i=0;i<M;++i)
{
    int a,b;
    cin >> a >> b;
    adj[a][b] = 1;
    // if the graph directed remove next line
    adj[b][a] = 1;
}
```

Space complexity $O(N \cdot N)$

Weighted Adjacency Matrix

graph have a value in it's edge

```

int N,M;    cin >> N >> M;
vector<vector<ll>> adj(M+1,vector<ll> (M+1));
for (int i=0;i<M;++i)
{
    int a,b, value;
    cin >> a >> b >> value;
    adj[a][b] = value;
    // if the graph directed remove next line
    adj[b][a] = value;
}

```

BFS (Breadth-First Search)

```

vector<int> BFS(vector<vector<int>> &adj) {

    vector<int> bfs;
    queue<int> Q;
    vector<int> vis(adj.size());
    Q.push(0);
    vis[0] = 1;

    while(!Q.empty()) {
        bfs.push_back(Q.front());

        for (auto i: adj[Q.front()]) {
            if (vis[i] != 1) {
                Q.push(i);
                vis[i] = 1;
            }
        }
        Q.pop();
    }

    return bfs;
}

```

Time Complexity $\rightarrow O(N) + O(2 \cdot E) \rightarrow E$ is a total number of edge

DFS (Depth First Search)

```
// Function to perform DFS traversal on the graph
vector<int> dfsOfGraph(int V, vector<int> adj[]) {
    vector<int> out;          // Output vector to store the DFS traversal
    bool vis[10000] = { 0 }; // Visited array to mark visited nodes

    // Start DFS from node 0 (assuming node 0 exists in the graph)
    dfs(0, vis, out, adj);

    return out;
}

// Recursive DFS function
void dfs(int node, bool vis[], vector<int>& out, vector<int> adj[]) {
    out.push_back(node); // Add the node to the output list
    vis[node] = true;    // Mark the node as visited

    // Traverse all the adjacent nodes
    for (int neighbor : adj[node]) {
        if (!vis[neighbor]) {
            dfs(neighbor, vis, out, adj); // Recursive DFS call on the neighbor
        }
    }
}
```

Space Complexity $\rightarrow O(N) + O(N) + O(N) = O(N)$

Time Complexity $\rightarrow O(N) + O(2 \cdot E)$

this code will give you all nodes when you go from parent to smallest child

Sieve of Eratosthenes

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; i++) {
    if (is_prime[i] && (long long)i * i <= n) {
```

```

        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}

```

$O(N)$

Different types of subs

Subarray

the subarrays of array {1, 2, 3} are {1}, {1, 2}, {1, 2, 3}, {2}, {2, 3}, and {3}.

Substring

string 'apple' are 'apple', 'appl', 'pple', 'app', 'ppl', 'ple', 'ap', 'pp', 'pl', 'le', 'a', 'p', 'l', 'e', ''.

Subsequence

{nums, B, D} is a subsequence of sequence {nums, B, C, D, E} obtained after removing {C} and {E}.

Subset

{3, 1} is a valid subset of {1, 2, 3, 4, 5}, but it is neither a subsequence nor a subarray.

Ordered set

You have to include some lines.

```

#include<bits/stdc++.h>
using namespace std;
typedef long long ll;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

typedef tree<
int,
null_type,
less<int>,

```

```

rb_tree_tag,
tree_order_statistics_node_update>
ordered_set;
/*
All the above are important to use ordered_set
*/
//Use less_equal<int> instad of less<int> for ordered-multiset

int main(){
    ordered_set oss;

    //insert value
    oss.insert(b);

    //get the size
    ll len=oss.size();

    //get index of value
    ll pos=oss.order_of_key(val);

    //get value of the index
    ll val=*oss.find_by_order(pos);

    //erase value by position
    oss.erase(oss.find_by_order(pos));

    //find and delete value
    ll pos=oss.order_of_key(a);
    ll val=*oss.find_by_order(pos);
    if(a==val)oss.erase(oss.find_by_order(pos));

    return 0;
}

```

Ordered Map

```

#include <iostream>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

template <typename K, typename V>
using ordered_map = tree<K, V, less<K>, rb_tree_tag, tree_order_statistics_no

int main() {
    ordered_map<int, int> omap;

    // insert key-value pairs
    omap.insert({3, 30});
    omap.insert({1, 10});
    omap.insert({4, 40});
    omap.insert({2, 20});

    // get the size
    cout << "Size of map: " << omap.size() << endl;

    // access value of a key
    cout << "Value of key 2: " << omap[2] << endl;

    // erase a key-value pair by key
    omap.erase(3);

    // find and erase a key-value pair by key
    auto it = omap.find(2);
    if (it != omap.end()) omap.erase(it);

    // check if a key exists
    cout << "Key 4 exists: " << (omap.find(4) != omap.end()) << endl;

    // get the index of a key
    cout << "Index of key 1: " << omap.order_of_key(1) << endl;

```

```
// get the key at a particular index
cout << "Key at index 2: " << (*omap.find_by_order(2)).first << endl;

// get the value at a particular index
cout << "Value at index 2: " << (*omap.find_by_order(2)).second << endl;

// clear the map
omap.clear();

return 0;
}
```